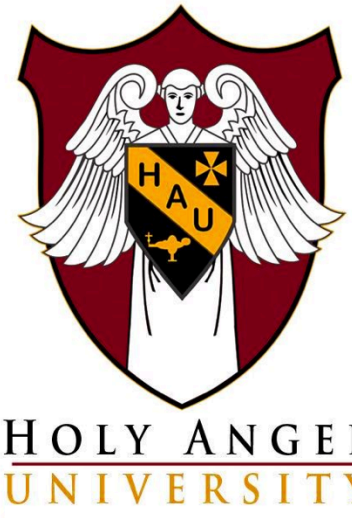
HOLY ANGEL UNIVERSITY

School of Computing
Holy Angel University
Angeles City

Final Project Proposal: PurrFile

A Primary Course Requirement in Applications Development and Emerging Technologies - 6ADET



Submitted by

Cayanan, Marcus Danford D.

Section:

CS - 301

Submitted to:

Mr. Tjakoen A. Stolk

6-ADET Professor

September 18, 2026

Date of Submission

HOLY ANGEL UNIVERSITY

1. **App Proposal**

Time: 60 minutes. Fill this in for your own idea. Write in full sentences where it says to - a proposal that only you can decipher next month isn't useful to future-you either.

App name

PurrFile

The app name remains unchanged from the initial proposal.

The problem, in one sentence

Cat owners often struggle to organize and access their cats' medical records, vaccination schedules, medications, and veterinary history because the information is scattered across paper records, clinic receipts, and personal notes, making long-term health management difficult.

Who is this for

Who, specifically, uses this? (Not "everyone" - name a person or small group, even if it's you.)

PurrFile is designed primarily for cat owners, especially those who have multiple cats, cats requiring regular medical care, or owners who want a more organized way to monitor their cats' health.

Currently, cat owners may rely on paper vaccination cards, veterinary receipts, handwritten notebooks, phone notes or calendar apps, spreadsheets, or simply their memory to keep track of their cats' health information and appointments. These methods can be difficult to organize and may result in misplaced records, missed vaccinations, forgotten medications, or incomplete medical histories.

PurrFile brings this information together into one centralized application where cat owners can manage their cats' profiles, health records, reminders, and general health questions.

What do they currently do instead of using your app (paper, memory, a different app, nothing)?
Currently, many cat owners rely on **paper vaccination cards, veterinary receipts, handwritten notebooks, their phone's notes or calendar apps, spreadsheets, or simply their memory** to keep track of important health records and appointments. These methods can be difficult to organize, easy to misplace, and may lead to missed vaccinations, forgotten medications, or incomplete medical histories.

PurrFile provides a centralized digital platform where users can securely store health records, receive timely reminders, and use the **AI Health Assistant** to access personalized health information and general pet care guidance based on their cat's medical records.

Core features (MVP)

List 3 to 5 features your app cannot ship without. For each, write one sentence describing what the user does and what happens.

Feature	Still in the MVP?	Flutter Pieces It Needs	Honest Estimate
1. Cat Profile Management	**Keep**	ListView.builder, Card, TextField, Form, Image, Navigator.push	**4 Hours**
2. Health Record Management	**Keep**	ListView.builder, Card, TextFormField, DropdownButtonFormField, showDatePicker, Navigator.push	**6 Hours**

3. Reminders & Notifications	**Keep**	ListView.builder, Card, showDatePicker, Switch, showDialog	**5 Hours**
4. AI Health Assistant	**Stretch**	TextField, ListView.builder, API/HTTP integration	**8+ Hours**

Estimated MVP Total: 15 hours

The MVP will focus on the three features that are essential to PurrFile's main purpose:

1. **Managing cat profiles**
2. **Managing and viewing health records**
3. **Managing health-related reminders**

The AI Health Assistant remains part of the project vision but is moved to the stretch goals so that the project remains achievable within the semester.

Stretch goals (optional, do later if there's time)

List features that would be nice, but the app works without them. Keep this list short - 2-3 items. These are what you build if the MVP goes faster than expected, not a second to-do list you're secretly planning on.

1. **AI Health Assistant** — Allows users to ask general questions about their cat's health and receive informational responses based on available health records.
2. **Downloadable Health Reports and Charts** — Allows users to generate PDF health reports and view simple charts such as weight history.
3. **Cloud Backup and Multi-Device Synchronization** — Allows users to securely access their PurrFile data from multiple devices.

NEW: How My App Saves Data

If two different people install my app, should they see the same data?

No.

Each user should have their own account and their own cat information. One user's cats, health records, and reminders should not be visible to another user.

This is important because PurrFile stores personal information about the user's cats and their health history.

Roughly how many records does my app hold in a realistic week of use?

For a typical cat owner, PurrFile would probably store approximately:

- **1–5 cat profiles**
- **5–15 health records per cat**
- **2–10 reminders**

A realistic user could therefore have around **20–40 records** during regular use.

The application is not expected to handle thousands of records for the MVP.

My Choice

Supabase

I plan to use **Supabase** as the persistent storage solution for PurrFile.

Why Supabase and Not the Others?

Supabase is appropriate for PurrFile because the app contains several related types of information, such as **users, cats, health records, and reminders**. A structured database allows

these records to be connected through IDs instead of storing everything as one large object.

The tradeoff is that Supabase requires more initial setup than simple local storage such as shared_preferences. However, PurrFile needs data to remain available after the application closes and should eventually support user-specific data, making a proper database more suitable for the project's direction.

What I Save, Concretely

Thing	Fields	Where it is Saved
User Account	User ID, name, email, password/authentication information	Supabase Authentication
Cat Profile	Cat ID, user ID, name, photo, breed, sex, date of birth, color, microchip number, owner name	cats table
Health Record	Record ID, cat ID, record type, date, description, veterinarian/clinic, notes, next due date	health_records table
Reminder	Reminder ID, cat ID, reminder type, reminder date, notification status	reminders table
AI Conversation	Conversation ID, cat ID, user question, AI response, date/time	ai_conversations table — only if AI is implemented

Have I Tried It Yet?

Not yet.

My first step will be to create a small one-hour Supabase test by connecting the Flutter application to Supabase, creating the `cats` table, and testing whether one cat profile can be saved and retrieved successfully.

If the test reveals that Supabase will take significantly more time than expected, I will reassess the storage approach before building the complete persistence system.

Target: August 27, 2026.

NEW: One Thing I Want to Add That the Course Did Not Teach

AI Health Assistant

The feature I would like to add is an **AI Health Assistant**.

The AI Health Assistant will allow cat owners to ask general questions about their cats' health and receive informational guidance. It may also summarize existing health records and help users understand information stored in PurrFile.

The initial proposal already describes the AI as an informational assistant that should recommend consulting a veterinarian when necessary and should not replace professional veterinary advice.

Package / Technology

I plan to use a **Flutter-compatible HTTP/API integration** to communicate with an AI service.

Where It Runs

The AI feature will be designed to work with the Flutter application. If the real AI service cannot be demonstrated reliably on the web during development, I will provide a **stand-in/mock response system** so that the AI Health Assistant screen can still open and function for demonstration purposes.

The actual AI interaction can then be demonstrated through a working device build or recorded demonstration.

Core Feature or Stretch Goal?

Stretch Goal.

The application does not depend on AI to solve its primary problem. Users can still create cat profiles, manage health records, and set reminders without the AI Health Assistant.

Therefore, the AI feature will only be implemented after the MVP is working.

NEW: How My Project Runs When Someone Else Opens It

- **I am keeping the device_preview wrapper: Yes.**
- **My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable: Yes, as the target for the final version.**
- **Anything that needs real hardware degrades to sample data instead of crashing: Not applicable to the core MVP.**
- **API keys/passwords/real personal data:** Any Supabase credentials or AI API keys will not be committed to the public repository. Development secrets will be stored locally using an ignored .env or equivalent configuration, while deployment secrets will be stored using repository/deployment secrets.

The project will remain in a **public GitHub repository under my own account**, while secrets and real personal information will remain outside the repository.

Data the App Remembers

The data list is updated to match the proposed Supabase storage structure.

Thing	Fields	Where it is Saved
User Account	User ID, name, email, password/authentication information	Supabase Authentication
Cat Profile	Cat ID, user ID, name, photo, breed, sex, date of birth, color, microchip number, owner name	cats table
Health Record	Record ID, cat ID, record type, date, description, veterinarian/clinic, notes, next due date	health_records table
Reminder	Reminder ID, cat ID, reminder type, reminder date, notification status	reminders table
AI Conversation	Conversation ID, cat ID, user question, AI response, date/time	ai_conversations table — only if AI is implemented

Screens

List every distinct screen this app needs. Aim for 3-6. If you're over 6, go back and cut a feature above.

1. Login / Sign Up
2. Dashboard (List of Cats)
3. Cat Profile *(includes AI Health Assistant)*
4. Health Timeline
5. Add / Edit Health Record
6. Reminders & Notifications

Stretch Screen

7. **AI Health Assistant Chat**

Scope Decision

Although the assignment recommends **3–6 screens**, the AI Health Assistant is considered a **stretch feature**, so the six screens above represent the actual MVP.

The AI Health Assistant can remain in the wireframes as an optional seventh screen because it is part of the project's planned extension.

Risks, Revised

The risk I named last time

AI Health Assistant implementation remains a risk, but it is now smaller because I have moved it from the MVP to a stretch goal.

In the initial proposal, the AI Health Assistant was identified as the most challenging part because it needs to provide useful and context-aware responses while clearly communicating that it does not replace professional veterinary care.

Moving it to the stretch goal reduces the risk of it preventing the core application from being completed.

First Step

Build the AI chat interface using **mock responses first**, without connecting to an actual AI service.

Target Date

After the MVP is functional.

A New Risk I Did Not See Before

The new risk is persistent data storage and connecting the Flutter application to Supabase.

PurrFile now needs to save and retrieve multiple related types of data: users, cats, health records, and reminders. If the database structure or Flutter-Supabase connection is implemented incorrectly, it could affect several screens at once.

First Step

Create a small Supabase prototype that can:

1. Connect Flutter to Supabase
2. Create one cat profile
3. Save the profile
4. Retrieve the profile
5. Display it in Flutter

Target Date

August 28, 2026

What Changed, and Why

Section	Prelim Said	Now Says	Why It Changed
Core Features	Four main features: Cat Profile Management, Health Record Management, Reminders & Notifications, and AI Health Assistant.	Three features are the MVP; AI Health Assistant is now a stretch goal.	The AI feature requires external API integration and additional testing. Moving it to stretch protects the completion of the core application.
AI Health Assistant	Included as an MVP feature that provides summaries, explanations, answers, and recommendations.	Moved to a stretch goal.	The core purpose of PurrFile can still be achieved without AI, so it should not risk the semester project's completion.
Data Storage	The preliminary proposal listed the information the app needs to remember but did not specify a concrete database implementation.	Supabase is the planned persistent storage solution.	Version 2 requires a specific persistence decision and an explanation of the tradeoff.
Data Structure	User Account, Cat Profile, Health Record, Reminder, and AI Conversation were listed.	These are mapped to specific database tables/collections.	The revised proposal must show where every saved field will actually live.
Screens	Six main screens were listed, with AI Health Assistant included through the Cat Profile.	Six screens form the MVP, while AI Health Assistant is treated as an optional seventh screen.	The project needs to stay within the recommended MVP scope while preserving the planned AI extension.
Risk	The biggest concern was implementing a useful and safe AI	AI remains a stretch risk, while Supabase/database	The scope change reduces the impact of AI, while persistent

	Health Assistant.	integration is the new main technical risk.	data becomes more important to the actual application.
Scope	The project planned to include AI as part of the main functionality.	The MVP focuses on cat profiles, health records, and reminders.	These features directly solve the original problem and are more realistic to complete within the semester.

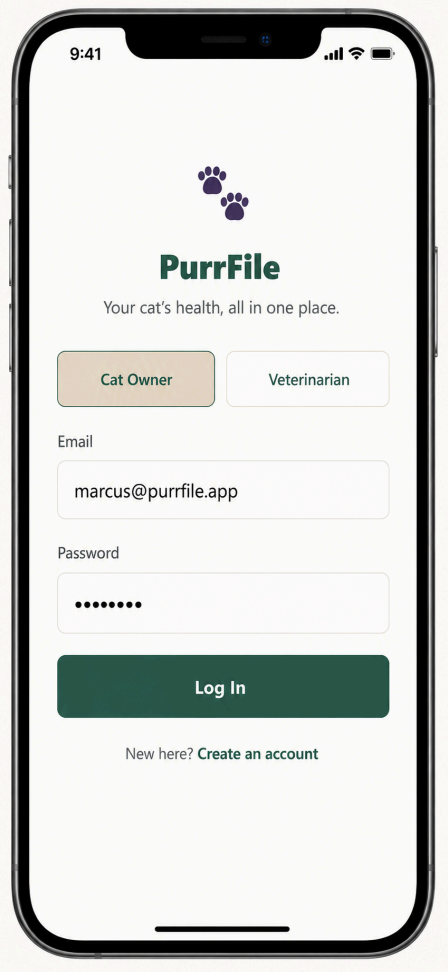
Overall, PurrFile's main purpose remains focused on helping cat owners organize and manage their cats' health information in one place. The revision mainly focuses on simplifying the MVP, moving the AI Health Assistant and other advanced features to stretch goals, choosing a suitable way to store user data, and addressing the technical challenges identified during development.

2. High-level mockup

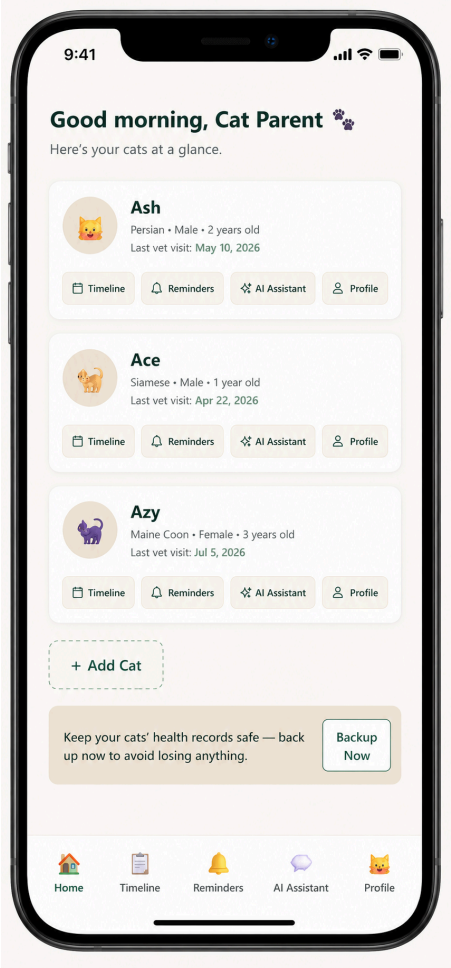
Your wireframes are done. You drew them for the prelim and they answered what goes where: boxes, labels, and the path from one screen to the next. Leave them alone. This is the next layer, not a redo. A mockup answers a different question: what will this actually look like? Same screens, same layout, but now with your real colors, your real type, real words instead of "label here", and icons where icons go. It is the picture you would show someone to explain your app without opening it. You are handing in images. A PDF of your screens, or PNGs, exported from whatever tool you like. Not a description of a mockup. The picture is the deliverable.

Step A and Step B: Purrfile Screens

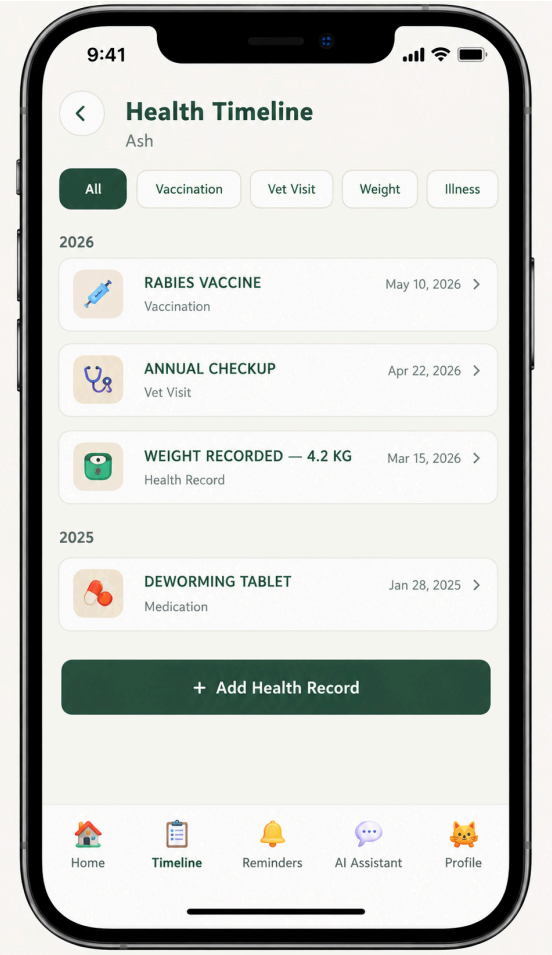
1. Login/Sign up



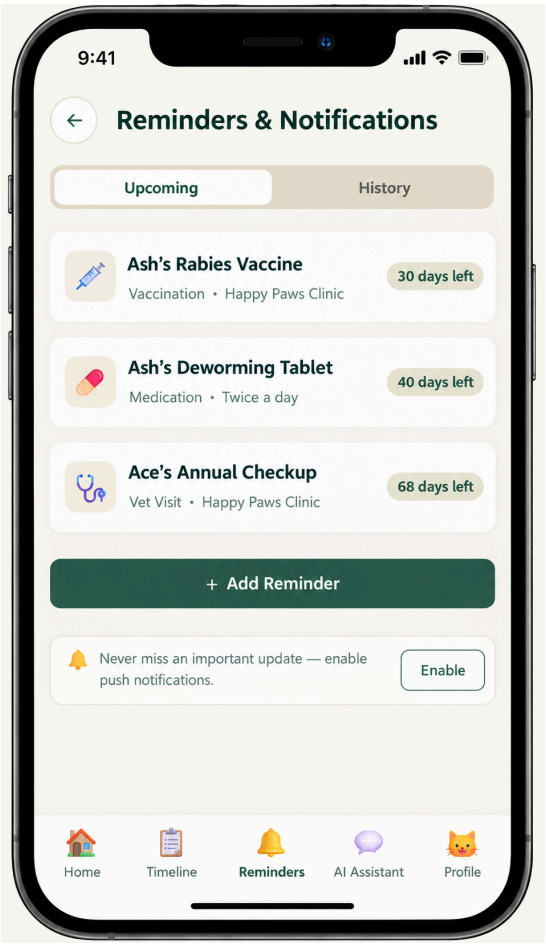
2. Home/Dashboard



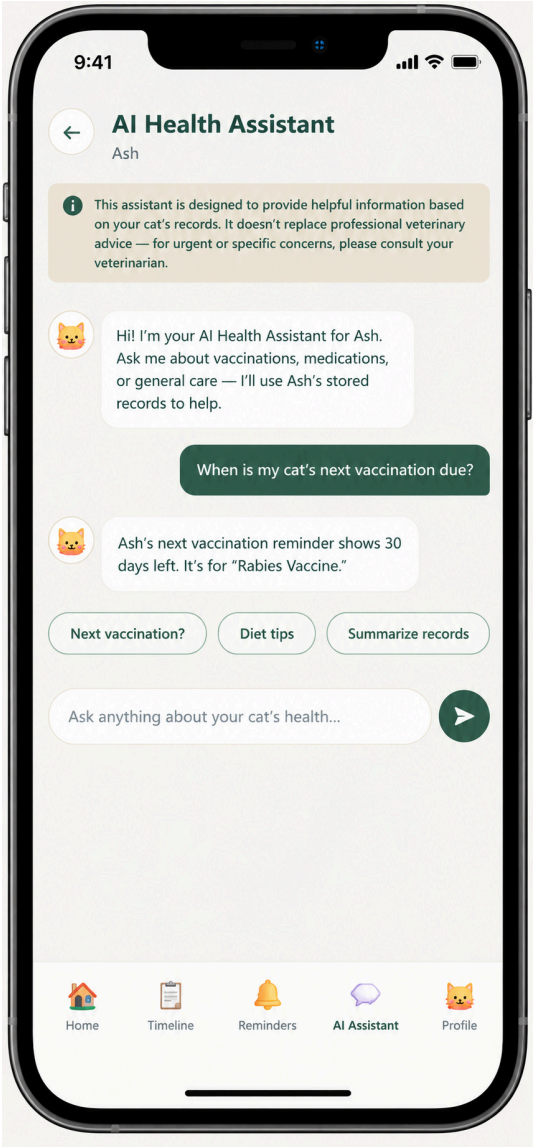
3. Timeline



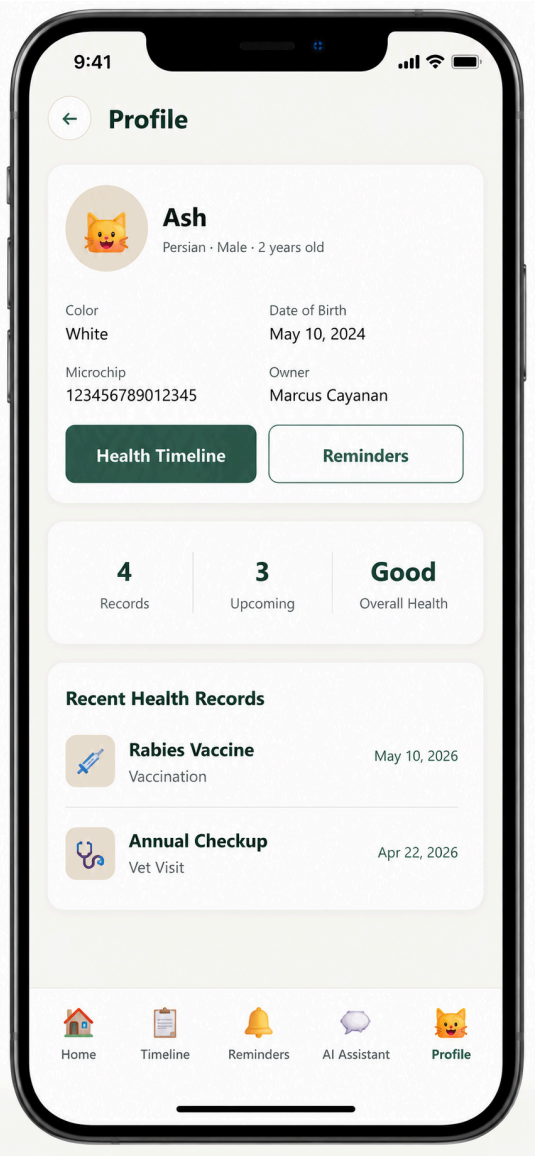
4. Reminders & Notifications



5. AI Assistant

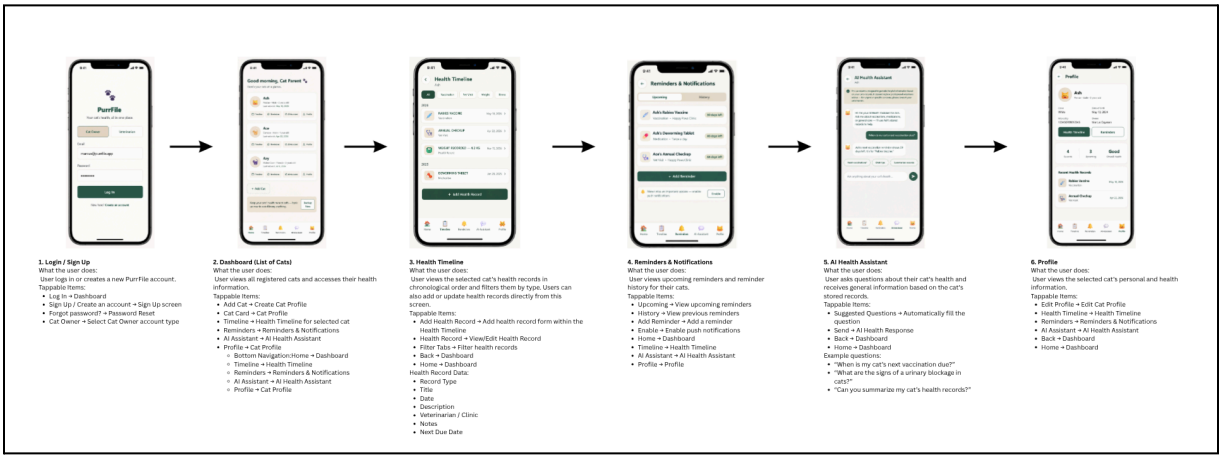


6. Profile



Step C and Step D: Label and In order

Purrfile - High Level Mockup



Canva Link (Mockup & System Design) : https://canva.link/enni8d7ynloatfb

What Changed and Why

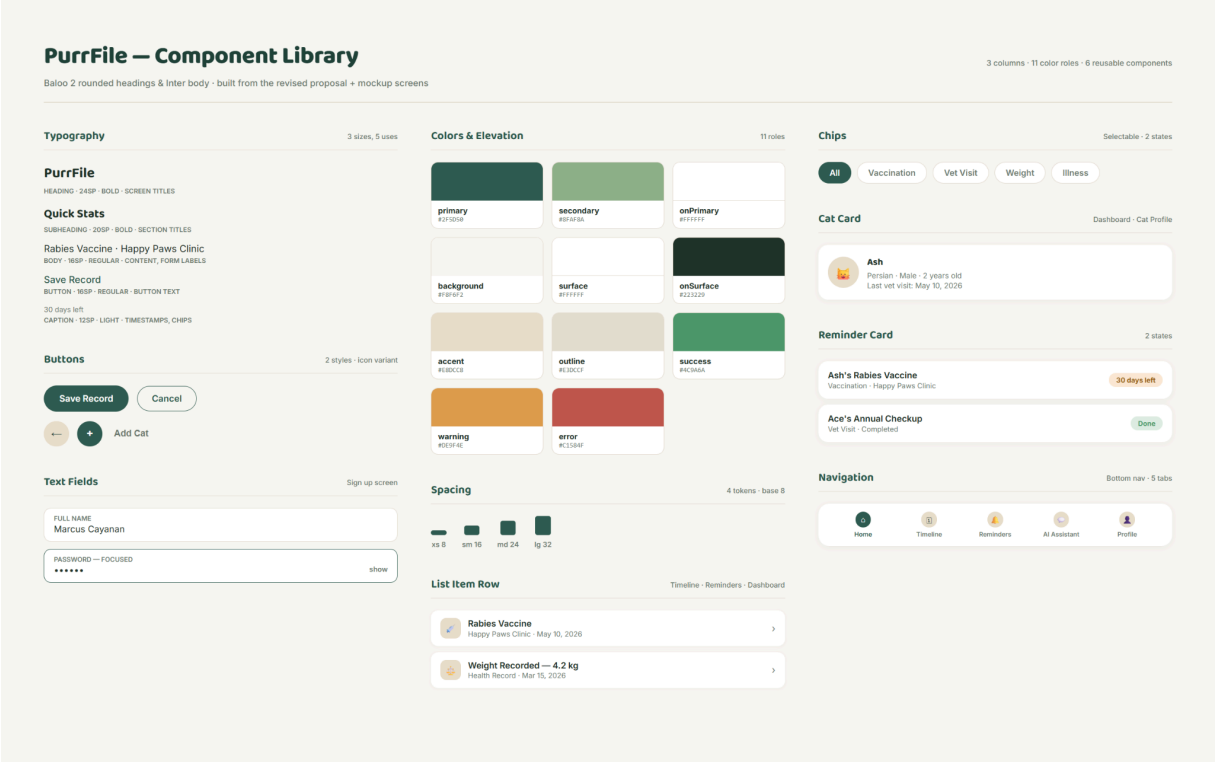
Your wireframes stay as they were. This section is about what you learned while painting them in, because that is where plans usually break.

Screen or element	The wireframe assumed	The mockup shows	What I will change
Dashboard	**Shows the user's registered cats and main features.**	**Cat cards now have direct shortcuts to Timeline, Reminders, AI Assistant, and Profile.**	**Keep these shortcuts for easier navigation.**
Health Timeline	**The timeline displays health records chronologically and allows users to filter records.**	**Records are grouped by year with filter options and an Add Health Record button.**	**Keep Add Health Record inside the Timeline instead of making it a separate main screen.**
Reminders & Notifications	**Users can view upcoming reminders, history, and add new reminders.**	**The mockup shows Upcoming and History tabs, reminder cards, an**	**Keep reminders as their own screen and allow users to manage reminders directly**

		Add Reminder button, and notification settings.	**from there.**
AI Health Assistant	**The AI assistant allows users to ask questions using their cat's stored health information.**	**The mockup shows an information notice, chat messages, suggested questions, and a message input field.**	**Keep the AI Assistant as a separate screen and make its responses based on the selected cat's stored records.**
Profile	**The profile is accessible from the dashboard and navigation.**	**The mockup places Profile as the last item in the bottom navigation and shows the selected cat's information, health summary, and recent records.**	**Keep Profile as the last screen/item in the main navigation and use it mainly for viewing the selected cat's information.**
Overall Screen Flow	**Uses separate screens for the main features.**	**The mockup shows six main screens: Login, Dashboard, Health Timeline, Reminders & Notifications, AI Health Assistant, and Profile.**	**Keep these as the main screens and simplify extra screens where possible.**

3. Design System, version 2

Last time you picked colors on paper. You have since written a real ThemeData in m4a3 and shipped two graded apps. This pass turns your design system into code you can paste.



Step A: The palette, as a ColorScheme (10 min)

Write your palette as the Flutter object it becomes. Fill in real hex values.

```
final scheme = ColorScheme.fromSeed(
  seedColor: const Color(0xFF6750A4),   // your primary
  brightness: Brightness.light,
);
```

ColorScheme.fromSeed generates a full, contrast-checked Material 3 palette from one color. Use it unless you have a reason not to, and if you do have a reason, write the full scheme out:

Role	Hex	Used for
primary	#2F5D50	Log In / Save / Add Reminder buttons, filled chip, "Ash's Rabies Vaccine" bot bubble, active nav label
onPrimary	#FFFFFF	Text/icons on top of primary (button labels, user chat bubble text)
secondary	#8FAF8A	Icon tints, sage badge fill, secondary highlights
onSecondary	#1F2E27	Text sitting on secondary (e.g. badge label)
background	#F8F6F2	Screen background on every screen
onBackground	#1F2E27	Body copy directly on the screen background
surface	#FFFFFF	Cards, list rows, text fields, chat bubbles (AI side)
onSurface	#1F2E27	Titles and body text on cards/surfaces
tertiary	#E8DCC8	"Cat Owner" tab fill, Upcoming/History tab bar, disclaimer banner, record-type icon chips
onTertiary	#5C4A2F	Text/icons on tertiary fills
outline	#E3DCCF	Card borders, unselected chip borders, text field borders
error	#C1584F	Reserved for overdue reminders / form validation (not yet shown in your mockup, but you'll need it once a vaccine goes past due)

Two checks the prelim did not ask for:

1. **Contrast.** *Put your body text color on your background color and look at it on a phone, outdoors if you can. If you generated with fromSeed, this is handled; if you hand-picked, check with a contrast tool (https://webaim.org/resources/contrastchecker/, aim for 4.5:1 on body text).*
   Body text (onSurface #1F2E27) on background (#F8F6F2) ≈ 13.2:1. Button text (onPrimary #FFFFFF) on primary (#2F5D50) ≈ 7.5:1. Both clear the 4.5:1 minimum for body text.

2. *Dark mode: yes or no, decided now. It is one extra ColorScheme with brightness: Brightness.dark plus darkTheme: on your MaterialApp, and it is much cheaper to decide now than to retrofit after you have hardcoded Colors.white in fifteen places. If the answer is no, write "light only" and then never hardcode a color anywhere except this file.*
   All six mockup screens use warm cream/beige surfaces; no dark variant is planned. Colors are only ever referenced through Theme.of(context).colorScheme, never hardcoded, so this is one extra ColorScheme and a darkTheme: line to add later if needed — not a retrofit.

Step B: The type scale, as a TextTheme (10 min)

Your style	Flutter slot	Size	Weight	Used for
Heading	headlineSmall	24sp	Bold	"PurrFile", "Health Timeline", "AI Health Assistant", "Reminders & Notifications"
Subheading	titleLarge	20sp	Bold	"Recent Health Records", stat row labels area
Body	bodyLarge	16sp	Regular	Record titles ("Rabies Vaccine"), form labels, AI response text
Button	labelLarge	16sp	Medium/500	"Log In", "Save Record", "Add Reminder"
Caption	bodySmall	12sp	Regular	"30 days left", clinic name + date subtitles, chip labels

Used by name, never by literal size:

Text('PurrFile', style: Theme.of(context).textTheme.headlineSmall)

That line is the whole point of a design system. Every TextStyle(fontSize: 24) you write inline is a decision you will have to find and change later. Custom fonts: google_fonts puts any Google font into your theme in one line, no asset files. See page 11. One font family is plenty.

Step C: Spacing, as constants

```
class AppSpacing {
  static const double xs = 4;
  static const double sm = 8;
  static const double md = 16;
  static const double lg = 24;
}
```

Base unit: *8px*

Screen edge padding: 16px (md) — matches the left/right margin around cards on every screen

Gap between list items: 12–16px — the space between "Ash's Rabies Vaccine" and "Ash's Deworming Tablet" rows

Gap between sections: 24px (lg) — the space between the reminder list and the "Add Reminder" button, and between "2026" and "2025" date groups on the Timeline

Step D: Component, as files

Component	File	Constructor parameters	Appears On
CatCard	**lib/widgets/cat_card.dart**	**Cat cat, VoidCallback onTap**	**Dashboard**
HealthRecordTile	**lib/widgets/health_record_tile.dart**	**HealthRecord record, VoidCallback onTap**	**Health Timeline, Profile ("Recent Health Records")**
ReminderTile	**lib/widgets/reminder_tile.dart**	**Reminder reminder, VoidCallback onTap**	**Reminders & Notifications**

PrimaryButton	**lib/widgets/primary_button.dart**	**String label, VoidCallback? onPressed**	**Login, Timeline, Reminders, Add/Edit Health Record**
FilterChipBar	**lib/widgets/filter_chip_bar.dart**	**List<String> options, String selected, ValueChanged<String> onSelected**	**Health Timeline ("All / Vaccination / Vet Visit / Weight / Illness")**
StatusBadge	**lib/widgets/status_badge.dart**	**String label, BadgeStyle style (upcoming / done / overdue)**	**Reminder cards**
ChatBubble	**lib/widgets/chat_bubble.dart**	**String message, bool isUser**	**AI Health Assistant**
PurrBottomNav	**lib/widgets/purr_bottom_nav.dart**	**int currentIndex, ValueChanged<int> onTap**	**every screen except Login**

All eight take data and callbacks, never setState, and use const wherever the constructor allows.

What changed, and why

Element	Prelim Said	Now says	Why it changed
Palette	**5 hand-picked colors, no contrast check done**	**Full hand-picked ColorScheme with 12 roles, contrast-verified**	**Once mockups existed I could sample real pixel values instead of guessing hex codes, and the caption gray actually failed 4.5:1 until darkened**

4. Midterm Reflection Journal

A. Module 4: Making a Screen

In Module 4, I started understanding how Flutter turns code into an actual screen. A **widget** is basically a building block of the app. Everything I see on the screen, such as text, buttons, images, and layouts, can be a widget. The build method is where I tell Flutter what those widgets should look like and how they should be arranged.

For layout, **Column and Row** took me the longest to get used to, especially when I needed different elements to fit properly on the screen. I sometimes had problems with spacing and alignment. It became easier when I started thinking of the screen as smaller sections instead of trying to arrange everything at once. I also learned that Expanded is useful when I want a widget to take the remaining available space.

In m4a3, I learned that theming is useful because I do not have to decide the colors and styles separately on every screen. I can set a consistent look and reuse it. This helped me when designing PurrFile because I wanted the screens to have the same dark green and beige style.

For m4a4, I was proud that I was able to create a complete HAUDEX screen instead of just a simple Flutter example. It helped me understand how different widgets work together. If I were to redo it, I would plan the layout first and make the code more organized before adding more details.

B. Module 5: Making It Move

StatefulWidget and setState helped me understand why changing a variable does not automatically change what I see on the screen. Flutter needs to know that something changed so it can rebuild the affected part. setState basically tells Flutter that the screen needs to update.

With input, TextField and controllers were useful because they allowed me to get information from the user instead of only displaying fixed text. I learned that the controller is what lets me access what the user entered.

I also learned why ListView.builder is better for lists than putting everything inside one Column. It makes more sense when there are multiple items because the list can build the items as needed.

Navigation was also something I had to practice. I used Navigator.push to move to another screen and pop to return. Passing data between screens was confusing at first, especially when the next screen needed information about a specific monster. Working on HAUDEX helped me understand that the selected object can be passed to the next screen instead of creating the data again.

For m5a5 HAUDEX, I would estimate that it took me around **6 to 8 hours** in total. Most of the time went into fixing the layout, connecting screens, passing data, and debugging rather than writing the first version of the code.

C. What Clicked, and What Is Still Shaky

One thing that finally clicked for me was **passing data between screens**. Working on the HAUDEX detail screens made me understand why the selected monster needs to be passed when navigating.

One thing I still find a little difficult is knowing when to use setState and how state should be organized when multiple widgets depend on the same data.

One bug I was proud of solving was when a screen or list was not showing the expected data. I checked the navigation and the values being passed instead of immediately rewriting the whole screen. This helped me realize that debugging is often about finding where the data stops being passed correctly.

An error message I now recognize is **"RenderFlex overflowed."** I usually understand it as a layout problem where the widgets are taking more space than what is available.

D. How I Work

When I get stuck now, I usually check my code first, read the error message, and test the problem in smaller parts. This is different from the prelim because I am now more comfortable searching for the specific part that is causing the problem.

I also use **AI tools** to help explain errors, suggest code, and give me examples. They are helpful when I do not know where to start, but I have also experienced getting code that does not work with my existing project. Because of that, I try to understand the code before using it instead of just copying everything.

I have also learned that leaving activities too late makes debugging more stressful because small errors can take longer than expected. For the final project, I want to keep working on tasks earlier and test each screen before moving to the next one.

E. Looking Forward

My final project, **PurrFile**, will rely heavily on what I learned from Module 5, especially for the **Health Timeline**. The HAUDEX activities taught me how to create lists, handle state, and navigate while passing data.

The biggest gap between what I have built and what PurrFile needs is **saving data permanently**. My previous activities mainly focused on the interface and temporary data. For PurrFile, I plan to use **Supabase** so cat profiles, health records, and reminders can be saved and retrieved.

By the end of the project, I want to be able to say that I can build a Flutter app that not only looks complete but also **saves and retrieves real user data**.

Two weeks from now, I want to have the main PurrFile screens connected and working in Flutter, including the Dashboard, Health Timeline, Reminders, AI Assistant, and Profile navigation.